Linked List Problem Bank

```
Legend
★ = Core Placement Problem (Mandatory)
Language-Agnostic DSA Problem Bank | Implement Node and List from scratch. Do NOT use
built-in list/array structures.
```

Section 1: Singly Linked List — Internals

★ **Create a Node Structure (Data + Next Pointer)**
★ **Create a Singly Linked List and Display (Traverse)**
★ **Insert a Node at the Beginning (Head)**
★ **Insert a Node at the End (Tail)**
★ **Insert a Node at a Given Position**
★ **Delete a Node from the Beginning**
★ **Delete a Node from the End**
★ **Delete a Node at a Given Position**
★ **Delete a Node by Value**
★ **Update Value of a Node at Given Position**
Count Total Nodes in a Singly Linked List
Search for a Value in the Linked List

Section 2: Doubly Linked List — Internals

★ **Create a Node with Prev and Next Pointers**
★ **Create a Doubly Linked List and Traverse (Forward and Backward)**
★ **Insert at Beginning, End, and Given Position**
★ **Delete from Beginning, End, and Given Position**
Convert Singly Linked List to Doubly Linked List

Section 3: Circular Linked List — Internals

★ **Create a Circular Singly Linked List and Traverse**
★ **Insert a Node in a Circular Singly Linked List**
★ **Delete a Node from a Circular Singly Linked List**
★ **Create a Circular Doubly Linked List and Traverse**
★ **Insert and Delete in a Circular Doubly Linked List**
Convert Singly Linked List to Circular Linked List

Section 4: Reversal Problems

★ **Reverse a Singly Linked List (Iterative)**
★ **Reverse a Singly Linked List (Recursive)**
Reverse a Doubly Linked List
★ **Reverse a Linked List in Groups of K**
Reverse Nodes between Two Given Positions

Section 5: Two Pointer / Slow-Fast Pointer

★ **Find the Middle Node of a Linked List**
★ **Find the Nth Node from the End**
★ **Detect a Cycle in a Linked List (Floyd's Algorithm)**
★ **Find the Starting Node of a Cycle**
Check if a Linked List is a Palindrome
Find Intersection Point of Two Linked Lists

Section 6: Merge and Sort Problems

★ **Merge Two Sorted Linked Lists**
★ **Sort a Linked List using Merge Sort**
Merge K Sorted Linked Lists
Sort a Linked List of 0s, 1s, and 2s
Add Two Numbers Represented as Linked Lists

Section 7: Advanced Linked List Patterns

★ **Remove Duplicates from a Sorted Linked List**
Remove Duplicates from an Unsorted Linked List
★ **Delete N Nodes after Every M Nodes**
Flatten a Multilevel Doubly Linked List
★ **Clone a Linked List with a Random Pointer**
Rotate a Linked List by K Positions
Segregate Even and Odd Nodes in a Linked List

Section 8: Real-World Applications of Linked List

★ **Polynomial Addition using Linked List (Each Node: Coefficient + Exponent)**
★ **Polynomial Multiplication using Linked List**
★ **Music Playlist using Doubly Linked List (Next / Previous / Shuffle)**
★ **Browser History using Doubly Linked List (Back and Forward Navigation)**
★ **Sparse Matrix Representation using Linked List**
Memory Free-List Simulation using Linked List (Memory Allocation / Deallocation)
★ **Josephus Circle Simulation using Circular Linked List**
Student Record Management using Linked List (Insert by Roll Number Order)
Train Coach Management — Insert and Delete Coaches using Linked List
★ **Undo / Redo in a Text Editor using Doubly Linked List**